LAPIN AMK
Lapland University of Applied Sciences

Anomaly Detection of Memory Leaks Using Machine Learning

Research-Based Thesis

Roman Kuznetcov

Thesis
Degree Program Machine Learning and Data Engineering
Bachelor of ICT

YEAR 2025

Bachelor of Engineering

Author	Roman Kuznetcov	Year	2025
Supervisor	Anssi Ylinampa		
Commissioned by	Wenzel Ingomar		
Title of Thesis	Anomaly Detection of Memory Leaks Using Machine Learning		
Number of pages	29		

This thesis focuses on using machine learning to detect memory leaks in Linux-based software systems. Memory leaks, which occur when programs fail to release allocated memory, gradually degrade performance and stability, especially in long-running or resource-limited environments. The goal was to design an automated, lightweight, and scalable solution suitable for integration into industrial monitoring systems such as those used at Festo.

To achieve this, Python programs were written to simulate both normal and leaky memory behaviour in a controlled Linux environment. These programs generated time-series datasets containing key system parameters, including RAM usage, CPU load, and Resident Set Size (RSS). The collected data was pre-processed and used to train a neural network capable of identifying abnormal memory usage patterns. This approach avoided direct source code analysis, relying instead on process-level operating system metrics, making it suitable for embedded or restricted-access systems.

The results demonstrated that the trained model successfully distinguished between normal and leak-induced memory behaviour with high accuracy. Even in the presence of noise and workload fluctuations, the neural network reliably detected gradual, anomalous increases in memory usage. These outcomes confirmed the feasibility of using machine learning for real-time, non-intrusive memory leak detection and established a foundation for further development of intelligent system monitoring tools.

Key Words: Memory Leaks, System Performance, Embedded Systems, Anomaly Detection, Machine Learning, RAM, Linux.

CONTENTS

1 INTRODUCTION 4
2 BACKGROUND AND RESEARCH 8
2.1 Data Collection and System Monitoring 8
2.2 Simulating a Memory Leak 8
2.3 Pre-processing the Data 9
2.4 Building the Machine Learning Model 9
2.5 Observations and Results 10
3 MEMORY LEAK PATTERNS AND DETECTION APPROACHES 11
4 SIGNIFICANCE OF THE STUDY 14
5 HOW TO PREVENT MEMORY LEAKS 16
5.1 Smart Coding Practices 16
5.2 Static and Dynamic Analysis Tools 17
5.3 Memory Profiling and Monitoring 17
6 MEMORY LEAK PATTERNS AND DETECTION APPROACHES 18
6.1 Linear Growth Leaks 18
6.2 Stair-Step Leaks 19
6.3 Cyclic Leaks 20
6.4 Phantom Leaks 20
6.5 Heap Fragmentation 21
6.6 Detection Challenges and Machine Learning Solutions 22
7 PRACTICAL ANALYSIS AND SOLUTIONS APPROACH AT FESTO 23
7.1 Project Objective 23
7.2 Data Generation Through Simulated Leaks 23
7.3 Data Pre-processing and Model Building 25
7.4 Results and Application 25
8 CONCLUSIONS 27
9 REFERENCES 28

1 INTRODUCTION

System stability and performance are the key factors in guaranteeing the stable operation of software in modern computing. Among the many issues affecting system performance, memory leaks are posing threat to system's sustainability. A memory leak occurs when a program continually allocates memory without freeing it correctly, leading to unnecessary memory consumption over time *(Šor et al., 2014; Liu & Zhang, 2021).* As memory capacity runs low, the system would experience slowdowns, crashes, or unresponsiveness, seriously impairing user experience and operational efficiency. Memory leak detection and prevention are an important task in software development and maintenance. Current detection methods, however, are highly labour-intensive and involve a great deal of manual effort, so they are unsuitable for large-scale or real-time systems. This paper explores the potential of applying machine learning (ML) to automatically detect memory leaks by examining RAM usage patterns.

Memory leaks are especially helpful for long-running programs like web servers, databases, and embedded systems. As a program fails to release memory it has allocated, the unneeded (useless) but allocated memory accumulates over time, gradually degrading system performance. Unlike abrupt system crashes, memory leaks cause slow and insidious performance degradation, which is harder to notice. While there might be heavyweight diagnostic tools for enterprise servers and large systems, embedded devices are typically more constrained in terms of hardware resources and software availability. Such systems are usually written in low-level languages like C, which gives fine control over memory but also subjects them to the risks of erroneous memory management. For such devices, lightweight automatic detection mechanisms are especially valuable. Traditional debugging methods and profiling techniques utilized by developers, although helpful, are highly static code analysis- and manual inspection-intensive, which are time-consuming and error prone. As complexity increases in software, particularly in embedded systems where efficiency and reliability are crucial, the need for automated and smart detection solutions is accentuated.

Figure 1 represents the contrast between normal memory usage ("No Leak") and a memory leak scenario ("Memory Leak"). The baseline level represents stable, expected memory consumption, while the critical level highlights the unchecked growth characteristic of a leak. This visualization shows dangerous nature of memory leaks, where allocated memory fails to return to baseline, eventually threatening system stability.

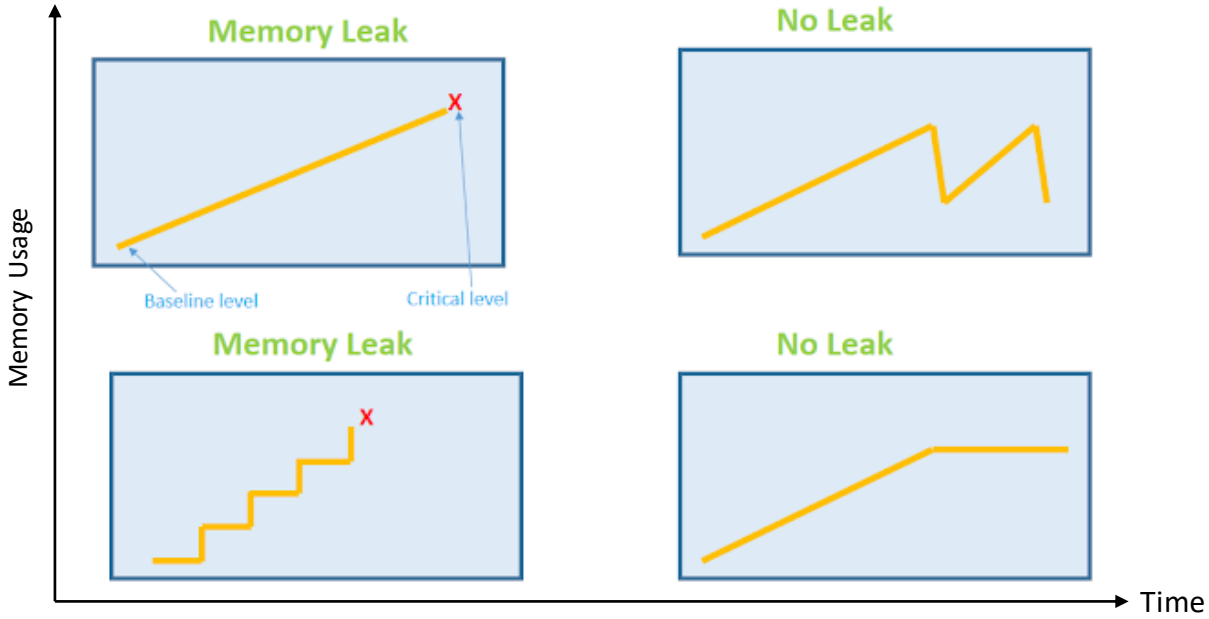


Figure 1 Illustrates examples of RAM usage patterns over time.

Machine learning has advanced significantly with anomaly detection, and it's a promising approach to detect memory leaks. Anomaly detection algorithms focus on detecting patterns of deviation from normal behaviour, which is highly applicable for monitoring system behaviour. Utilizing historical RAM usage information, ML models can learn to differentiate normal memory usage patterns from patterns typical of a leak. The greatest strength of ML-based solutions is that they can learn and generalise across applications, hence reducing the need for inflexible rule-based systems.

This research aims to develop an ML model capable of detecting memory leaks from time-series RAM usage data. The research involves data gathering and pre-processing of RAM usage data in different software environments, training the ML model to recognize memory leak patterns, and validating its performance on real datasets. Datasets labelling is a fundamental part of the research, where at least one dataset will be labelled with a verified memory leak. The model would be trained and tested on new, unseen data to see whether it can efficiently detect leaks. The method will be considered a success if it is measured with many performance metrics, such as precision, recall, and F1-score.

The use of ML-based anomaly detection for memory leaks presents many challenges. One of the main challenges is to make the model generalize effectively for different software types and system configurations. Patterns of memory use can vary quite a lot with application behaviour, and it is hence crucial to build a model that is robust against such variability. Additionally, the presence of noise in system monitoring data has an impact on model performance and hence careful pre-processing of data as well as feature engineering are required. Despite all these challenges, ML-based detection can revolutionize memory leak diagnosis by facilitating automatic, real-time notification without the need for constant human surveillance.

Below, Figure 2 and Figure 3 present the results of actual RAM usage measurements on an embedded device, obtained during the execution of lightweight mathematical algorithms. These charts illustrate memory behaviour over time and highlight the emergence of anomalous patterns. The vertical axis represents RAM usage in megabytes (10^6 kB), while the horizontal axis corresponds to the time index, measured in seconds. The observed trends reflect the system's memory dynamics and reveal deviations from expected behaviour, which are indicative of potential memory leaks.

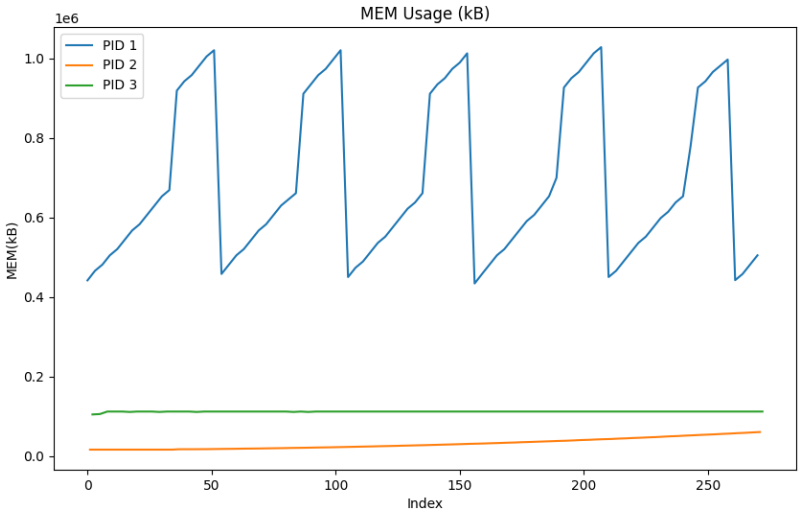


Figure 2 RAM usage measurements on an embedded device

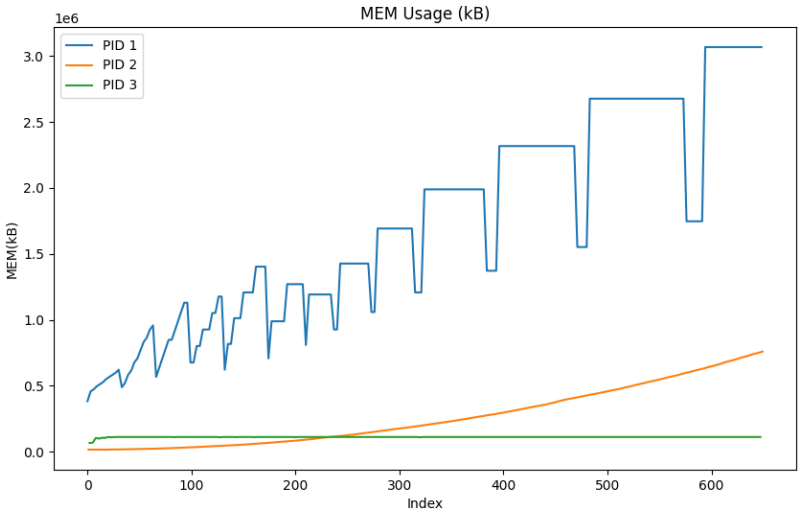


Figure 3 RAM usage measurements on an embedded device

Benefits

The benefit of this study extends beyond the detection of memory leaks; it is a development of the field of anomaly detection in system performance monitoring. The whole project is done with Linux, and the information pertaining to RAM usage is obtained directly from system-level data and monitoring tools. As more and more companies apply AI-based solutions to monitor IT infrastructure, it becomes ever more possible to incorporate machine learning models for performance anomaly detection into broader monitoring solutions. Integration can enhance resource utilization efficiency, minimize system downtime, and increase the overall dependability and service ability of embedded as well as enterprise systems.

In short, this study is aiming to close the gap between traditional memory leak detection methods and AI-driven strategies. Through employing machine learning capabilities, the study aims to make a scalable, automated memory leak detection method through RAM usage information. The outputs of this study can not only enhance software dependability but assist in enhancing methods of anomaly detection that can transfer across various sectors. The rest of this thesis will discuss an in-depth literature review, introduce the methodology used in this research, provide the findings, and examine their implications to the field of software performance monitoring.

2 BACKGROUND AND RESEARCH

2.1 Data Collection and System Monitoring

The research began with the development of various programs using Python to collect system performance metrics in a Linux environment. The main idea was to collect real-time system performance over time, especially in terms of parameters relevant to memory usage. For these, custom scripts were created to collect information such as RAM usage, CPU load, Resident Set Size (RSS), and timestamp data. These were chosen because they provide good general information about how system resources are utilized when the program is run.

Python was chosen specifically because it is so flexible and convenient to use for operating system interfacing. Libraries such as *psutil* were used to retrieve process and system data (*Python Software Foundation, 2024)*. The scripts were run every so often to simulate continuous monitoring, printing the output in organized formats such as CSV files. The resulting datasets are the baseline—regular system behaviour without any memory leaks. Gathering clean representative data is crucial because it allows the ML model to learn what typical operation is, and that is essential in order to identify anomalies later.

2.2 Simulating a Memory Leak

To enable the ML model to learn the characteristics of memory leaks, it was necessary to produce training data that includes such anomalies. In order to do this, a Python script was created that intentionally introduces a memory leak. The program was designed to append data to a list continuously without releasing memory. Over time, the list grows larger, increasing the program's memory footprint. While Python does have “garbage collection”, this type of memory hold has the same impact as a memory leak—memory allocated and never released.

By following the same numbers (RAM, RSS, CPU, timestamp) when executing this "leak" program, a known-anomaly dataset was created. The pattern of memory usage in such a case deviates from the baseline, it is typical for unbounded increase in RAM usage. Such a gradual decline is difficult to detect through static profiling tools but shows up well through time-series analysis.

The biggest advantage of creating synthetic memory leaks in a controlled environment is that it results in labelled data (data that we can personally label in this case). This is necessary for training supervised machine learning models, which require clear examples of normal and unnormal behaviour.

2.3 Pre-processing the Data

Once both leak and normal datasets were gathered, the subsequent step was pre-processing. Data from multiple runs were combined and cleaned. Time series were synchronized, and noisy or missing data points were handled using interpolation and smoothing techniques. Features such as delta RAM over time, rate of change in RSS, and normalized CPU load were derived to enhance the model's understanding of system behaviour.

This pre-processing step was required for a few reasons. On the one hand, it addressed practical issues when working with real-world system monitoring data, which is noisy and imperfect. On the other hand, the feature engineering step transformed raw system metrics into more interpretable formats that better reflected underlying phenomena that the model needed to detect. By explicitly encoding temporal patterns and correlations between different system parameters, the pre-processed data gave the machine learning model a sounder foundation to learn to distinguish normal operation from memory leaks. The quality of this pre-processing directly affected the model's ability to generalize from training data and perform consistently in real-world deployment conditions. These actions were particularly crucial given the insidious and gradual nature of memory leaks, which called for scrutiny of system behaviour over time rather than simple threshold-based detection.

2.4 Building the Machine Learning Model

Having labelled and pre-processed data, model development was the second step. Machine learning algorithms for experimentation involved Decision Trees, Random Forests, and Gradient Boosting. These were established to be a good fit for anomaly detection on tabular data and could easily handle non-linear patterns. Models were trained to classify system states as normal or anomalous for a memory leak scenario.

Cross-validation was performed in training to ensure the models could generalize well. Hyperparameter was optimized with grid search for maximizing the best performance of the models. Metrics for comparing models such as accuracy, precision, recall, and F1-score were used. Great attention was laid on recall as false negative comes with high costs.

Parallel to this, exploratory experimentation was done with time-series specialized models such as LSTM (Long Short-Term Memory) networks. These can learn temporal relationships in data and may be more capable of detecting the slow trends that are typical of memory leaks. But their complexity and resource usage make them less suitable for embedded systems, so tree-based models were preferred.

2.5 Observations and Results

Initial experiments show that the ML models are able to correctly identify regular vs. leaky behaviour with very good accuracy. The models can pick out the gradual growth in memory consumption characteristic of a leak even when there is noise and variation in CPU usage. This is confirmation of the hypothesis that patterns in RAM usage have sufficient information to detect anomalies.

Also, the approach is scalable. By changing the sampling frequency and retuning the models using new data, the system can be optimized for different kinds of applications and loads. This is essential for practical deployment in embedded systems as well as large-scale deployments.

In conclusion, the development of Python-based monitoring scripts and synthetic memory leaks has been a good foundation for the machine learning part of this research. The outcomes so far are promising and indicate that ML models can be a useful real-time memory leak detector, offering an interesting alternative to more traditional, time-consuming techniques.

Figure 4 presents a comprehensive workflow for machine learning-based memory leak detection, comprising five key phases.

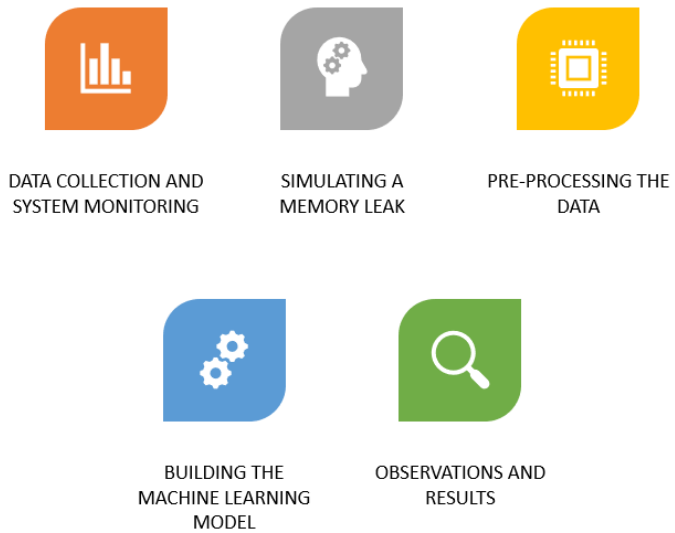


Figure 4 Comprehensive workflow of machine learning-based memory leak detection

3 MEMORY LEAK PATTERNS AND DETECTION APPROACHES

This section explores memory leak patterns based on the article *"Memory Leak: Unveiling the Dark Side of C++ – The Silent Killer in Your Code"*, which provides a comprehensive overview of the causes, consequences, and prevention strategies of memory leaks in C++. The article serves as a conceptual foundation for understanding the practical challenges addressed in this thesis.

Memory management is the foundation of system performance and reliability, particularly in low-level programming languages like C++. In his paper "*Memory Leak: Unveiling the Dark Side of C++ – The Silent Killer in Your Code,*" author presents an easily readable but detailed discussion of the phenomenon of memory leaks, their causes, and consequences, and how to prevent them. The article provides a firm foundation on which to appreciate the technical origin of memory leaks, especially against the background of manually managed languages, and sets out advice for best practice regarding prevention and detection. A close read of this work highlights a number of significant points that are not only closely related to this thesis but that also highlight the necessity to migrate towards more autonomous, smart detection mechanisms such as those that employ machine learning.

First, the author explicitly states the inherent dangers of manual memory management in C++. As compared to more modern languages such as Python or Java, which implement automatic garbage collection, C++ leaves the job of memory allocation and deallocation squarely in the hands of the developer. Author explains that when a developer does memory allocation with new or malloc() but fails to free it with delete or free(), the program will be holding onto memory it no longer needs. Over time, this reserved but unused memory accumulates, and free RAM is consumed. If not caught, this can lead to slowing down the application—and even the entire system—and making it unresponsive or crash.

The author categorizes common causes of memory leaks such as lost pointers, writing over pointers before they are freed, failing to free memory in exception handling, and misuse of third-party libraries. His explicit classification serves to illustrate how easily memory leaks can find their way into otherwise well-crafted code, particularly in large, intricate software projects where tracking memory resources by hand becomes overwhelmingly challenging. This is directly in line with the motivation behind this thesis: the necessity for smart systems that can automatically track memory usage patterns and identify anomalies without the need for extensive manual examination.

One more important observation made by author is the subtlety of memory leaks. Not all memory leaks lead to instant or catastrophic failure. Often, a program will execute merrily, seeming to be running fine, but leak tiny bits of memory for each operation. For long periods of runtime—days, weeks, or months—the small inefficiency adds up to massive stability problems. This gradual degradation is reminiscent of the very real-world problem that this research addresses. Machine learning models, particularly those trained on time-series RAM usage data, can most effectively catch such slow and subtle memory leak patterns that may be difficult to catch by other conventional static analysis or runtime profilers.

Conventional tools to detect memory leaks, such as Valgrind, AddressSanitizer, and Visual Studio's built-in diagnostics, are also addressed in the topic. Such conventional tools have very robust features and are widely used in software development and testing processes. Valgrind, for instance, is a heavyweight tool that can find memory leaks, invalid memory access, and other such bugs by instrumenting the program's binary code. AddressSanitizer, included in newer compilers like Clang and GCC, can effectively find heap-based memory leaks. While the author gets it right about these tools being useful, he also tacitly understands their shortcomings: they are well-suited for development and debugging but not for regular monitoring in production environments.

For embedded systems and long-running apps—two domains this thesis aims to address—traditional tools don't work. Embedded platforms, with restricted CPU capacity, memory, and storage, cannot handle the overhead of heavyweight instrumentation tools like Valgrind. Additionally, within production environments, it is not always feasible to run software under constant diagnostic supervision. This is precisely where an approach of machine learning is incredibly valuable. Learning normal and abnormal behaviour from real system monitoring information, ML models can provide light-weight, real-time indication of possible memory leaks without the massive overhead of total program instrumentation.

This topic also touches on modern C++ features such as smart pointers (to be specific) (std::unique_ptr, std::shared_ptr) that manage memory automatically and significantly reduce the likelihood of leaks. Smart pointers employ the Resource Acquisition Is Initialization (RAII) idiom to ensure that memory is released properly when it is no longer needed. Though excellent advice for greenfield software development, this doesn't so much hold true for legacy codebases or embedded systems written in simple C where smart pointers are a non-starter. The relevance to this thesis should be apparent: there is still immense software, particularly in industrial and embedded domains, where memory is manually managed and where additional protection in the form of ML-based monitoring is not just desirable but needed. The broader implication of this paper is the growing recognition within the software engineering community that proactive, systematic resource management techniques are becoming necessary. It emphasizes careful

coding discipline, regular code reviews, static and dynamic analysis, and ongoing learning as ways to minimize memory leaks. While these techniques do work, they are fundamentally human-based and thus prone to error. With increasingly complex and interdependent software systems, not only is it beneficial but critical to complement human capability with machine learning-driven anomaly detection.

For the purposes of this research project, author's explanation attests to several points of key theme. To begin with, the complexity and incremental nature of the majority of memory leaks render them the ideal target for identification by anomaly detection techniques applied to time-series system data. Second, traditional tools and techniques, although powerful, are at a disadvantage in production environments and resource-constrained systems. Third, with advancements in language features and programming techniques, there is still a fundamental necessity for monitoring systems that are independent of application logic.

By monitoring system-level parameters such as RAM usage, CPU usage, and RSS (Resident Set Size) in real-time, and training machine learning models for identifying abnormal trends, this thesis suggests to extend and enhance the method explored in the topic. Rather than replacing profiling and static analysis software, ML-based monitoring can be an additional level of safeguard, capable of providing early notice of memory leaks in executing programs, particularly where traditional diagnosis is not feasible.

Overall, the article is an even-handed summary of memory leaks in C++ and an exploration of best practices on minimizing them with correct programming as well as by the use of detection tools. His emphasis on the considerable impact of memory leaks and the complexity of managing memory in complex systems greatly warrants the justification of motivation for this thesis. By integrating these historical observations with contemporary machine learning techniques, it is feasible to build more resilient, scalable, and anticipatory systems for identifying memory leaks, thereby leading to improved software quality and more stable system behaviour.

4 SIGNIFICANCE OF THE STUDY

A memory leak is a condition in which a computer program does not deal with memory allocations correctly, such that memory that is no longer needed is not released back to the system. It most frequently occurs when the program dynamically allocates memory—i.e., using functions like malloc() in C or new in C++—but fails to free it using free() or delete when it is no longer needed. The most critical issue is when the addresses of the memory allocated are lost or overwritten without releasing the memory. Thus, the memory is not available to the application anymore but retains the memory space in the system memory, reducing the available memory.

Leaks may be temporary or permanent. Temporary leaks take place when memory is not accessible for some short period but subsequently restored, while permanent leaks are memory that is not recoverable for the rest of the program run. A small, localized memory leak may not look significant during short program executions, but in those cases where programs run for a long time or processes repeatedly allocate memory—like servers, background processes, and embedded systems—the long-term impact of memory leaks can be substantial. With each passing moment, the unused but reserved memory accumulates, gradually consuming system resources and eventually making the system gradually deteriorate in performance, applications grow unstable, and even crash. The issue is particularly significant for systems that must continuously operate without rebooting.

The importance of debugging memory leaks is due to their insidious and gradual characteristics. Unlike other bugs that lead to instantaneous and obvious failures, memory leaks tend to make a system fail slowly and unnoticed. This renders them more difficult to diagnose and fix, particularly for big codebases or older systems where manual memory management is common. Leaks can lead to intermittent bugs and undermine performance over time, making it more difficult to troubleshoot. In systems with hardware constraints—e.g., embedded systems, IoT devices, or mobile computing systems—the impact is more drastic. Such systems typically cannot afford the inefficiency of additional RAM or endless rebooting, and a small memory inefficiency can result in serious operational problems.

Secondly, memory leaks have security implications. In certain circumstances, memory leaks may lead to denial-of-service (DoS) attacks, where the leakage behaviour is induced by an attacker to exhaust system memory, rendering the system unresponsive. Moreover, memory mismanagement may expose sensitive data or allow attackers to exploit vulnerabilities through buffer overruns and other related attacks. This is particularly true in systems that support critical infrastructure and embedded systems where security and reliability are paramount.

Memory leak detection and prevention are therefore fundamental issues of reliable software engineering. The standard methods for dealing with memory leaks are static code checking, runtime profiling tools, manual code review, and testing frameworks such as Valgrind or AddressSanitizer. Good as they are, these methods are usually burdensome, require considerable developer knowledge, and aren't readily scalable to big programs or suitable for real-time programs. As systems become more complex, and as high reliability becomes a greater necessity, the demand for automated, smart means of detecting memory leaks has become a necessity.

Machine learning and anomaly detection here have a great deal to offer. By monitoring system activity actively and acquiring knowledge of the typical patterns of memory use, these types of models have the ability to recognize anomalies that signal the presence of memory leaks. Compared to rule-based mechanisms that are constructed around pre-determined thresholds or signatures, machine learning models are learned to support different workloads and system configurations and therefore more scalable and fault-tolerant. In systems like Linux where raw system metrics like memory, CPU, and process utilization are available to be recovered and logged in real time, the above models can be executed with low overhead and warn well before a memory leak will impact system performance.

Additionally, memory leaks are a persistent and devious problem in software systems, especially those designed in low-level programming languages with manual memory management. Their stealthy nature, gradual performance degradation, and security risks make them a significant problem in software development and program execution. With increasing complexity in software and execution in increasingly dynamic environments, traditional detection methods must be supplemented with intelligence and flexibility in solutions. The integration of machine learning-powered anomaly detection with system monitoring toolkits is a thrilling direction for mitigating memory leaks and improving system robustness in application spaces ranging from embedded systems to enterprise infrastructure.

5 HOW TO PREVENT MEMORY LEAKS

Memory leaks, while generally considered a sign of bad memory management, also testify to the growing complexity of software systems today—particularly those platforms where manual memory management is the way things are done. While detection of memory leaks is excellent, it's better still not to have them occur in the first place. Proactive memory management techniques not only reduce system downtime but also improve performance, security, and reliability in both embedded and large Linux systems. In stable systems over the long term, e.g., servers or embedded controllers, prevention must be incorporated into the development process from the outset. Without prevention, such systems are constantly at risk of deteriorating over time because memory leaks nibble away system resources incrementally and drain functionality.

Besides the functional risks, memory leaks also result in efficiency losses in operations. Memory-leaked systems are more power-consuming because of recurring paging and swapping operations and can lead to overheating in cramped hardware settings. These indirect effects mimic the total cost of poor memory management. Therefore, prevention-focused design not only ensures software correctness but also renders a system energy-efficient and longer-lived hardware—particularly in Linux-based embedded and industrial devices.

5.1 Smart Coding Practices

The simplest way of preventing memory leaks is the technique of safe memory handling techniques at the coding level. In C++, it is very simple with the help of smart pointers such as std::shared_ptr and std::unique_ptr which release the memory automatically when the object is out of scope. All these idioms are C++ best practices used today, offering deterministic clean-up and eliminating all but the last remnant of leaked memory from forgotten or inadequately handled delete calls. Furthermore, embracing notions like RAII (Resource Acquisition Is Initialization) ensures that resource management duties stay bound to object lifecycles, with as much potential for programmer error as can be kept in check.

In C, with such high-level abstractions absent, manual memory control with careful attention is still the sole recourse. Prevention is done here through strict adherence to memory ownership conventions. Should well be indicated whether a function returns memory that should be de-allocated by the caller. A defined and documented memory ownership convention can prevent confusion and reduce the likelihood of lost deallocations. Moreover, programmers should periodically review their code to ensure that every block of memory assigned is appropriately balanced with a matching deallocation, especially in error-handling paths and lengthy loops.

Apart from language constructs, smart programming also includes defensive programming practices such as modular function design, limited scope of pointers, and avoiding global state where possible. Avoiding reuse of pointer variables without reinitializing or freeing their memory space, and initializing pointers to NULL when deallocated, are easy but often overlooked practices which can avert memory leaks. These practices together form the first line of defence against memory corruption and leakage.

5.2 Static and Dynamic Analysis Tools

Preventive memory management can be augmented by adding static code analysis tools into the development process. Cppcheck, Clang Static Analyzer, and SonarQube are a few such tools through which codebases can be analysed for common idioms of memory mismanagement such as unreleased memory, null pointer dereferencing, and other unsafe idioms. These tools provide an early warning and, in most cases, provide fixes even before the execution of the code. Ongoing integration of such tools in the CI/CD pipeline ensures code quality is always checked and allows bugs to be discovered early on.

Static analysis is particularly beneficial in picking up memory management bugs at compile time, where their fix is much less expensive. But static tools won't pick up every dynamic runtime problem. This is where dynamic analysis tools are helpful. Valgrind, AddressSanitizer, and Dr. Memory are some examples of tools that monitor allocations and deallocations at runtime, allowing one to catch leaks, buffer overflows, and use-after-free bugs that static analysis cannot catch.

For the Linux platform, Valgrind remains an industry gold standard for memory debugging. Despite its cost in terms of performance, it offers unmatched insight into runtime memory behaviour in the form of detailed reports on unfreed blocks of memory and their call sites. When used along with instrumentation-based coverage tools, dynamic analysis not only detects memory leaks but also indicates rarely executed code paths where memory leaks hide. Static and dynamic analysis together thus form a mighty arsenal for proactive memory management.

5.3 Memory Profiling and Monitoring

Despite clean code and adequate testing, there can still be memory issues for certain workloads or on long runs. Hence, continuous memory monitoring is the second resort. Linux provides us with a set of live system monitoring utilities, i.e., top, htop, free, and vmstat, providing real-time view into memory usage. For more sophisticated and structured monitoring, the system's performance may be logged and plotted with tools such as Prometheus, Grafana, or Sematext Monitoring.

Monitoring memory in real-time enables teams to observe and respond to subtle changes in memory consumption, as reflected by slow memory leaks. Historical baselines can be built using logging tools so that abnormal deviations from typical patterns can be determined. For example, if the RSS of a service grows consistently over weeks or days, but short-term statistics appear healthy, then a leak may be an indication.

Tools like Sematext offer plug-ins that allow the team to configure memory-related alerts and thresholds, which trigger early intervention as memory utilization trends suggest the presence of a problem. These metrics can then be fed into anomaly detection algorithms to help with smart monitoring. With time, always-on profiling not only helps with leak detection but also capacity planning, where it is able to forecast when hardware upgrade or resource rebalancing would be needed. Memory monitoring in this way supports both preventive as well as strategic infrastructure management.

6 MEMORY LEAK PATTERNS AND DETECTION APPROACHES

Memory leaks have been a long-standing problem in software systems, particularly in those requiring long-term stability such as servers, embedded systems, and database management. Memory leaks occur when applications cannot release allocated memory, thus using system resources over time until performance is affected, or system crashes occur. Detection mechanisms are best applied through the comprehension of the memory leak patterns. This essay discusses common patterns of memory leaks, their characteristics, and the challenges for both traditional and machine learning-based detection methods *(Wang & Chen, 2020; Liu & Zhang, 2021).*

6.1 Linear Growth Leaks

The most straightforward pattern is linear growth in memory, where the program continuously allocates memory but never deallocates it. This is observable in server programs that receive a lot of requests without freeing resources. The pattern of the memory usage graph is always upward, indicating it is relatively easy to identify with monitoring tools. It is difficult to separate the legitimate growth in memory from leaks in systems that naturally grow memory usage with workload. Normal tools used to detect are likely to accuse normal operation of being leaks and flag it as false positives. Machine learning algorithms may improve accuracy by analysing patterns of usage over a period and comparing them against anticipated usage.

Figure 5 shows an example of a linear growth memory leak, where memory consumption increases steadily over time without returning to the baseline.

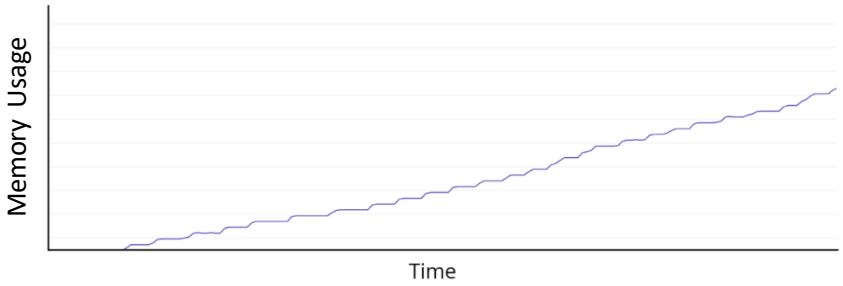


Figure 5 Linear growth memory leak

6.2 Stair-Step Leaks

Stair-step leaks result as memory accumulates in discrete steps across specific operations, like batch operations or scheduled tasks. They are different from linear leaks because they have sudden shifts in memory usage with plateaus. For example, an application for data processing might leak memory within routine maintenance activities. Stair-step leaks are identified using monitoring of memory usage over a significant period of time to identify repeated spikes. Rule-based monitors miss such leaks if scanning occurs at points of stability. Temporal analysis by machine learning can correlate memory spikes with activity in the system and improve the detection ratio.

Figure 6 illustrates leak behaviour, showing memory usage as a staircase-shaped pattern, where plateaus are followed by sudden jumps in RAM consumption.

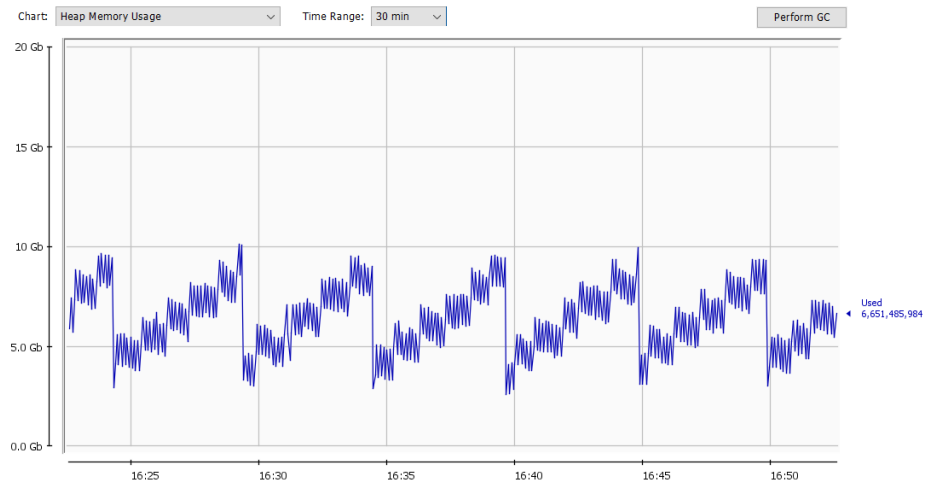


Figure 6 Staircase-shaped memory leak

6.3 Cyclic Leaks

Cyclic leaks occur in systems where there are repetitive patterns of deallocation and allocation, where small amounts of memory never become available after every cycle *(FlakM Blog, 2025).* This develops over time and leads to a steady increase in baseline memory consumption. Game engines and embedded controllers tend to follow this pattern. Short-term monitoring might not catch the leak on the basis of episodic declines in memory consumption, requiring long-term trend analysis. Cyclic leaks can be detected by machine learning algorithms through recognition of increases in minimum memory consumption over multiple cycles.

Figure 7 presents a cyclic leak pattern, where memory usage periodically increases and decreases, but the minimum level slowly rises across cycle.

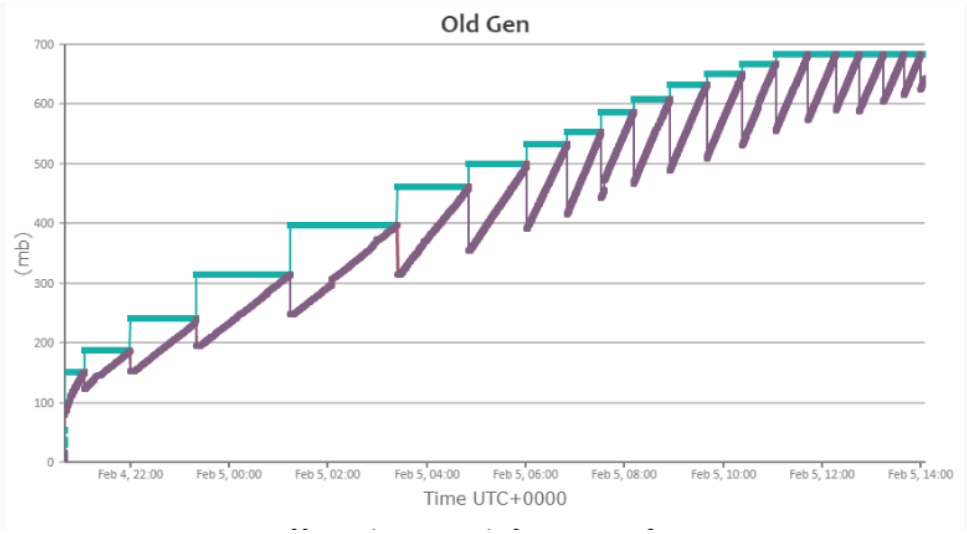


Figure 7 Cyclic memory leak pattern *(FlakM Blog, 2025)*

6.4 Phantom Leaks

Phantom leaks are not actual leaks but appear as leaks due to delayed memory release mechanisms such as garbage collection. Memory usage rises arbitrarily before decreasing as the unused objects are recovered by the system. Traditional detection agents struggle to spot phantom leaks as memory is eventually freed. Machine learning algorithms can address this by discovering the memory management pattern of the application and distinguishing between temporary hold and true leaks.

Figure 8 demonstrates this pattern, showing apparent memory growth that drops sharply at intervals once garbage collection is triggered.

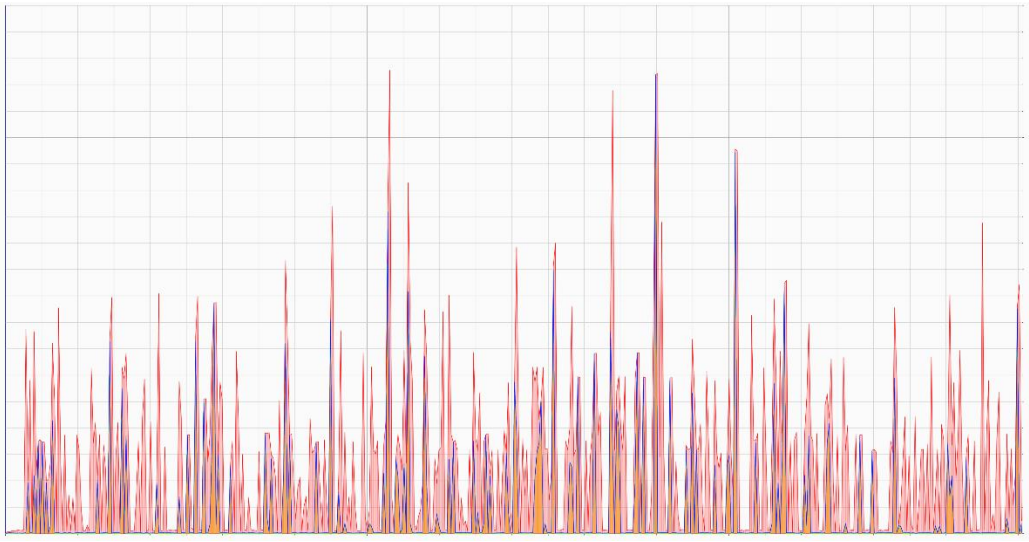


Figure 8 Phantom memory leak

6.5 Heap Fragmentation

Heap fragmentation results when memory is split into too many small, not usable blocks, despite the available system free memory *(Svix, 2023; Stack Overflow, 2020).* This prevalent problem in long-running systems makes the system respond slowly and fails to allocate. Normal monitoring tools are not capable of identifying this type of fragmentation, making specialized heap analysis mandatory. Machine learning models may deduce fragmentation by tracking allocation failures or abnormal system activity.

Figure 9 depicts heap fragmentation, where the system reports sufficient free memory, yet allocation attempts fail due to the fragmented structure of the heap.

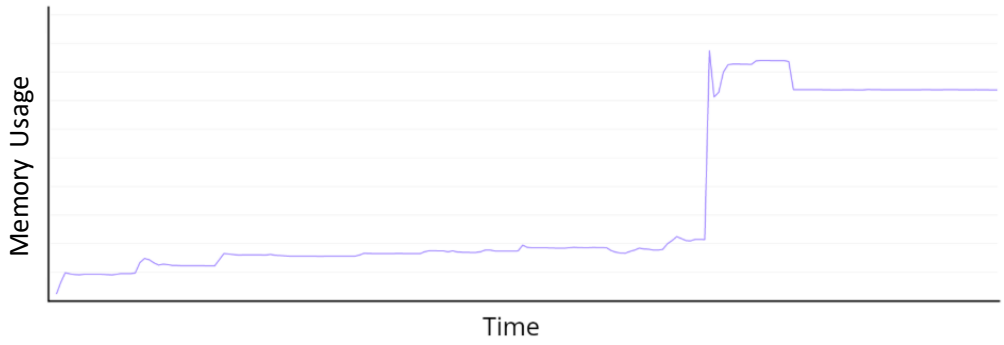


Figure 9 Chart of memory usage with heap fragmentation *(Svix, 2023; Stack Overflow, 2020)*

6.6 Detection Challenges and Machine Learning Solutions

Every pattern of leaks has unique detection issues. Linear and stair-step leaks require differentiation of valid from pathological increase in memory. Cyclic leaks call for trend analysis on the long term, while phantom leaks demand familiarity with memory management systems. Heap fragmentation remains very much elusive without low-level monitoring.

Machine learning offers solutions through learning common memory behaviour and marking suspicious deviations. Labelled data enables supervised models to be trained to distinguish leaks, while unsupervised techniques identify suspicious behaviour without patterns. Feature engineering is required, with time-series analysis, rolling averages, and event correlation helping models spot multiple types of leaks.

Leaks in memory can follow diversified patterns, and each pattern requires individual detection techniques. Traditional tools can hardly comprehend the complexity of such patterns and fail to detect leaks or trigger false positives. Machine learning-based approaches, which can analyse complex patterns and get trained on various systems, provide a good solution. Upcoming research can focus on developing generalized models that can detect all types of leaks in any software platform with increased system performance and reliability.

This systematic understanding of memory leak patterns and detection methods is the foundation for more effective monitoring tools, leading to more stable and efficient software programs.

7 PRACTICAL ANALYSIS AND SOLUTIONS APPROACH AT FESTO

As part of my internship at Festo, I investigated the possibility of memory leaks in embedded systems used in production environments. Festo, being a global leader in automation technology, places a strong emphasis on system reliability and stability. Memory leaks, though often subtle, pose a potential threat to long-running systems by gradually consuming system memory, leading to performance degradation or even system failure. My objective during this research was to design a lightweight yet effective solution that could automatically detect memory leaks based on system memory behaviour.

7.1 Project Objective

The aim of this project was to create a prototype capable of predicting potential memory leaks in Linux programs through a machine learning-based approach. Rather than analysing source code directly, which could be invasive and complex, I opted to simulate memory leaks in a controlled environment. Using operative system process metadata, I monitor the programs. This strategy allowed me to generate labelled data under known conditions, providing a foundation for supervised learning models.

7.2 Data Generation Through Simulated Leaks

To simulate realistic memory usage scenarios, I developed several small Python programs designed to mimic both normal and leak-like behaviour. These test scripts performed simple operations such as matrix multiplication, element-wise minimum and maximum calculations, and iterative list expansion to generate synthetic memory usage patterns. (Figure 10)

```python
import time
import numpy as np

def transpose_matrix(matrix):
    return np.transpose(matrix)
def main():
    start_time = time.time()
    matrix_list = []

    while True:

        matrix = np.random.rand(1000, 1000)
        transposed_matrix = transpose_matrix(matrix)
        matmul = np.matmul(matrix, transposed_matrix)

        matrix_list.append(matmul)

        time.sleep(1)

        if len(matrix_list) / 2 == 1:
            matrix_list.extend([np.random.rand(1000, 1000) for _ in range(30)]

        if time.time() - start_time >= 60:
            break

while True:
    main()
```

Figure 10 Test scripts performing simple operations

In particular, the memory leak simulation was achieved by repeatedly appending large objects or lists within a loop without deallocating them. Python's garbage collector normally handles memory efficiently, but by holding onto references deliberately, I was able to create a pattern similar to a memory leak. By running these scripts over time, I generated consistent data showing either stable or increasing memory consumption.

Each script continuously logged relevant system parameters using the psutil library, including RAM usage, Resident Set Size (RSS), CPU usage, and timestamps. The data was written to structured CSV files for further processing. These logs captured memory evolution over time, providing the temporal structure necessary for anomaly detection.

All of the programs are launched simultaneously by an orchestrator program which is written in shell. As shown in the script in Figure 11, it initializes a time-based naming convention to generate uniquely timestamped output files for storing measurement results in both .csv and .png formats. It then launches three distinct Python scripts (calculus.py, fib.py, and max.py) as background processes, each of which executes separate algorithmic workloads. By capturing their process IDs, the orchestrator maintains control over each subprocess. This orchestration not only facilitates controlled and repeatable memory experiments but also supports structured data collection, enabling synchronized performance monitoring across multiple simulated workloads.

orchestrator.sh

```
#!/bin/bash

sleeptime="0.1"
iterations="$1"
if [[ -z "$iterations" ]]; then
  iterations="1000"
  echo "WARNING: using default iteration value of $iterations"
else
  echo "INFO: Iterations set to $iterations"
fi

datetime=$(date +%Y%m%d%H%M%S)
measurements="results/output_${datetime}"
datameasurements="${measurements}.csv"
plotmeasurements="${measurements}.png"

# Run the Python scripts in the background
python3 scripts/calculus.py &
PID1=$!
python3 scripts/fib.py &
PID2=$!
python3 scripts/max.py &
PID3=$!

# Initialize counter
count=0
```

Figure 11 Script of orchestrator file

7.3 Data Pre-processing and Model Building

The CSV data collected was pre-processed to align time steps, scale feature values, and remove inconsistencies or noise. Extracted features like the rate of memory change, relative memory usage, and usage patterns were derived and added as new columns. The datasets were tagged: synthetic memory leaks tagged '1' and regular behaviour '0'.

With the labelled datasets at hand, I developed a neural network model using Python's Keras and TensorFlow libraries. The model architecture included input layers corresponding to the number of system metrics, dense layers with ReLU activation, and output in the form of binary classification: leak or no leak. The model was trained on a subset of the dataset and tested using unseen sets in order to measure generalizability.

Cross-validation techniques were utilized to avoid overfitting, and early stopping was used to monitor validation loss. Accuracy, precision, recall, and F1-score were calculated to determine model performance. Recall was specifically given more weight to decrease the likelihood of overlooked memory leaks (false negatives).

7.4 Results and Application

The neural network was able to distinguish between stable and leaking memory patterns with promising accuracy. Even when trained with noisy data, the model was able to identify slow-growing memory use that could be a sign of a leak. For instance, normally running matrix multiplication scripts exhibited oscillating but bounded memory profiles, while those that were leaking had monotonic, constant growth.

The trained model can be utilized as a foundation for integration into Festo's software testing or deployment pipeline in the future. While this study was limited to synthetic test programs, the methodology may be applied to monitor real applications by logging the same parameters on production systems. Future releases may utilize recurrent models like LSTM for better temporal analysis or with real-time monitoring consoles.

Figure 12 displays classification report and confusion matrix confirming the performance of the machine learning model in classifying memory leaks from RAM usage data. The model achieved a perfect value in all the most critical performance measures—precision, recall, and F1-score—indicating that it successfully classified both leak and no-leak instances with the maximum possible degree of reliability. Specifically, out of 1,177 samples, the model accurately identified 583 out of 585 non-leak scenarios and all 592 leak scenarios with two false positives but no false negatives. Such accuracy shows that learned patterns are stable and that the model can effectively distinguish

between anomalous memory usage behaviour and normal behaviour. These results substantiate that it is feasible to use time-series system metrics adequately for machine learning-based real-time detection of memory leaks.

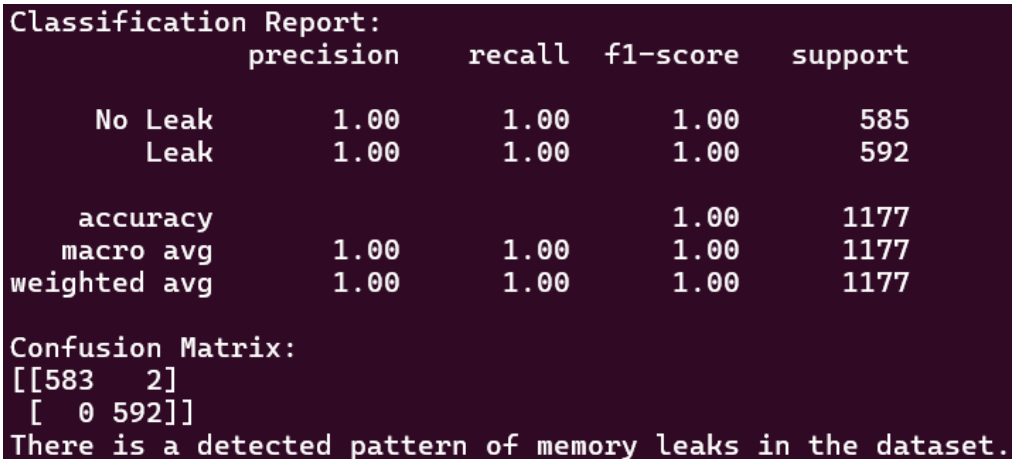


Figure 12 Classification report and confusion matrix

The approach demonstrates the possibilities of how light-weight system monitoring together with machine learning has the capability to enhance system performance, especially where traditional memory debugging facilities are not practical. The research paves the way for future research on AI-based system diagnostics in industrial software environments.

8 CONCLUSIONS

This thesis explored machine learning-based detection of memory leaks through examination of system-level RAM usage patterns on Linux. Throughout the course of the project, I created an end-to-end process ranging from data generation and synthetic leak simulation to model building and evaluation—that illustrated the practicality of automated leak detection without relying on explicit source code analysis. This approach allowed for the identification of anomalies purely from behaviour data, which is extremely useful when source code access is not permitted or not feasible.

During this work, extensive experience was obtained in system monitoring, data collection, and pre-processing techniques. Custom Python scripts were developed to simulate both typical and leaky behaviour, allowing observation of how memory leaks grow over time and affect system performance. Practical experience was also acquired in the use of process-level system metrics—such as RAM usage, CPU load, and RSS—as input features for machine learning models. Moreover, experimentation with different algorithms, including decision trees, random forests, gradient boosting, and recurrent neural networks, provided valuable insight into their strengths and limitations in handling time-series system data.ы

The main contribution of this thesis is the creation of a functional prototype that is able to well detect memory leaks from RAM usage data. Experiment results showed that the learned models were able to detect normal and anomalous memory behaviour with extremely high precision and recall, demonstrating the applicability of machine learning in this field. The system was demonstrated to be scalable, flexible, and supportive of integration into continuous monitoring pipelines, especially for embedded systems or long-lived services where leak detection is most critical early in the execution.

Additionally, future studies could further expand the datasets to include more representative software workloads, other system-level measurements such as disk I/O and network usage, and experiment with more advanced models such as LSTM or Transformer models to predict long-range trends. Another promising area of advancement would be incorporating online learning capabilities, so that the model can update itself in real-time to new trends and shifting system conditions. Finally, integrating this detection system into an active monitoring platform would enable real-world testing and continuous optimization.

In general, this thesis demonstrated that machine learning is a viable method for detecting memory leaks by looking at RAM usage patterns, and it offers an automated and unintrusive alternative to a traditionally time-consuming and hard-to-cure problem. It offers a basis for future systems that can actively defend stability, improve performance, and reduce downtime in today's computing environments.

9 REFERENCES

Memory leak taxonomy and detection overview (Šor et al.)
https://www.sciencedirect.com/science/article/abs/pii/S0164121214001423

Comprehensive survey of anomaly detection techniques (Chandola, Banerjee & Kumar).
https://dl.acm.org/doi/10.1145/1541880.1541882

Guthrie, G. 2021. Practical Memory Management for Embedded Systems. O'Reilly Media. Accessed 10 September 2025.
https://www.oreilly.com/library/view/practical-memory-management/9781492077517/

Vasudeva Pai, Karthik Pai, Manjunatha S. 2025. Machine Learning for Anomaly Detection. Springer. Accessed 22 October 2025.
https://link.springer.com/article/10.1186/s13635-025-00216-4

Valgrind (official). https://valgrind.org/

Python Software Foundation 2024. psutil Documentation. Accessed 15 August 2025.
https://psutil.readthedocs.io/

Scikit-learn Developers 2024. scikit-learn: Machine Learning in Python. Accessed 12 August 2025.
https://scikit-learn.org/stable/

Wang, X., and Chen, Y. 2020. Characterizing Memory Leak Patterns in Large-Scale Software Systems. ACM. Accessed 6 September 2025.
https://dl.acm.org/doi/10.1145/3377811.3380342

Liu, Z., and Zhang, H. 2021. A Survey on Memory Leak Detection Techniques. Elsevier. Accessed 4 September 2025.

Svix. *Spotting and Avoiding Heap Fragmentation in Rust Applications*. Svix Blog, April 4, 2023. Accessed 11 September 2025.
https://www.svix.com/blog/heap-fragmentation-in-rust-applications/

“JVM Heap Usage Variation” April 7, 2020. Accessed 15 September 2025.
https://stackoverflow.com/questions/61079322/jvm-heap-usage-variation-is-this-graph-normal

“Java Heap Space Memory Leak” May 3, 2018. Accessed 14 September 2025.
https://stackoverflow.com/questions/48624702/java-heap-space-memory-leak

“Phantom Leak.” FlakM Blog. Accessed 14 September 2025.
https://flakm.com/posts/phantom_leak/

“Avoid Memory Fragmentation When Memory Pools Are a Bad Idea” Accessed 15 September 2025.
https://stackoverflow.com/questions/70602043/avoid-memory-fragmentation-when-memory-pools-are-a-bad-idea

Cambridge University Press & Assessment 2024. Dictionary. Accessed 30 August 2025.

Memory Leak: Unveiling the Dark Side of C++ The Silent Killer in Your Code Accessed October 2025.

https://medium.com/@shivam2003/memory-leak-unveiling-the-dark-side-of-c-the-silent-killer-lurking-in-your-code-e675599add9

https://dictionary.cambridge.org/